

IoT technologies to enhance precision and response time of mobile-based educational assessments

Salsabeel Shapsough, Mahitab Hassan, Shams Eddeen Shapsough, Imran A. Zualkernan*

Department of Computer Science and Engineering

American University of Sharjah

Sharjah, United Arab Emirates

*izualkernan@aus.edu

Abstract—“Smart Education” is one key application domain for IoT. However, more often than not, the term smart education is used as a synonym for use of conventional e-learning systems like MOOCs, etc. IoT’s potential, however, goes far beyond the capabilities of current systems. IoT offers innovative ways of processing and collecting sensor data from distributed locations, network architectures to deliver the data effectively, and use of Big Data techniques to analyze this data for meaningful decision-making. This paper presents the design and implementation of a mobile/tablet-based assessment system that uses the MQTT publish-subscribe architecture to provide almost real-time assessment services to various types of educational stakeholders including teachers, principals, parents and educational planners. The system delivers and collects assessment data including sensor information from mobile devices used by teachers and students. The system routes analytics results generated from analytical engines to appropriate stakeholders in real-time. Preliminary analysis of the proposed system shows promising results in terms of low power consumption in mobile devices and good scalability.

Keywords— Smart education; Formative assessment; Real-time response; MQTT; R; IoT; Big Data

I. INTRODUCTION

Mobile devices/tablets are increasingly being used to conduct learning outcome assessments in many countries. For example, RTI’s Tangerine App [1] has been used to conduct standardized diagnostic learning assessments like Early Grade Reading Comprehension (EGRA) and Early Grade Mathematics Comprehension (EGMA) across the world. Each such educational assessment is an exam or a quiz consisting of multiple questions called ‘test items’ that either the teachers asks students (for younger ages), or the students attempt themselves. Such assessments are typically conducted at intervals of a few months to provide a diagnostics snapshot of the learning abilities and shortcomings of a student population and their teachers’ skills. Recently, there have been attempts to use similar mobile Apps for formative assessment as well [2]. In this study, assessment data from students was collected on a bi-weekly basis using a single mobile tablet provided to the teacher. At the end of the two week’s period, data from teacher’s device was synced with a centrally located Learning Management System (LMS) to provide reports to teachers and teacher mentors and trainers. This use of this data-driven approach resulted in significant learning effects [3]. However,

while the assessment data was being collected in an incremental and distributed fashion, the analysis and reporting of the assessment results was somewhat rigid and limited to bi-weekly reporting cycles. Similarly, a detailed analysis of the assessment data typically suffered from a further time-lag because the mobile assessment App was not directly integrated with analytical tools (e.g., R - <https://www.r-project.org/>). Finally, by its nature the granularity of analysis was restricted to assessment level. In other words, assessment data was collected and reported only after an assessment had been completed by all students in a class.

The primary goal of research presented in this paper is to explore the use of IoT technologies like the MQTT architecture to design and deploy a mobile-based assessment system that delivers flexible, on-demand, and almost real-time formative assessment analysis of assessment data collected using mobile devices in a highly asynchronous and distributed fashion. The system should allow stakeholders at various levels of an educational systems to dynamically subscribe and unsubscribe to relevant assessment reports. For example, micro-level reports [4] are pertinent for teachers, principals and parents who are concerned with school-level issues. Meso-level reports, on the other hand, are appropriate for school districts and relevant educational administrators. Finally, macro-level reports help formulate policy and provide visibility into teachers’ competency and training issues at a regional, provincial or country level. For example, at the micro-level a teacher should be able to subscribe to assessment reports related to a specific learning outcome (e.g., FRACTIONS for Fifth Grade Mathematics), and should be provided with real-time statistical benchmarking of their students against neighboring or similar schools. In addition, the teacher may, for example, subscribe to advanced analytic reports that guide them on how to do effective banding in their classroom (e.g., [5]). The reporting should not be restricted to a long assessment cycle (e.g., bi-weekly). This means that the moment a new mobile assessment item is attempted by a student in any part of the educational system, the results from this assessments are incorporated and immediately made available to the teacher. Similar modalities of reporting should be provided to other stakeholders in the educational system as well. Consequently, educational administrators at the regional level, for example, should not have to wait for the end of the

year to get a clear indication of teacher training gaps based on student learning outcomes.

This paper presents the design and implementation of a standards-based assessment system that uses mobile tablets to conduct assessments, is cross-platform, supports subscription and reporting in a real-time and incremental manner, and can be easily integrated into statistical and predictive analytic engines like R. Because the system also needs to be deployed in developing countries, the system also supports seamless off-line processing and device syncing for when Internet connectivity is unreliable or not available.

The rest of the paper is organized as follows. The architecture of the proposed system is presented next. This is followed by a preliminary performance analysis. The paper ends with a conclusion.

II. SYSTEM ARCHITECTURE

The system as shown in Fig. 1 consists of three distinctive modules: Instructor App, Student App, and the Core. Instructor and Student mobile Apps are built using the Apache Cordova cross-platform development framework (<https://cordova.apache.org/>). This ensures that Apps can be deployed on multiple hardware platforms including IOS, Android and Windows.

Storage services in App as well as the core are implemented using the NSQL database system called CouchDB [6]. The database stores assessments, student records, and analysis reports. Document-store databases like CouchDB allow for freedom and flexibility in the structure of documents where the entries can be of any size and structure as long as they are in the JSON format. Queries to this database are written as views using JavaScript and employ the map-reduce algorithm which works in two steps: the map function takes in the data or, in this case, the documents, and filters the input according to a certain condition. The reduce function, then takes the filtered output of the map function and groups the data in accordance to a prescribed criteria. The map-reduce functions improve the scalability and speed of retrieving documents from the database. Finally, CouchDB also supports master-to-master replication that allows all peer nodes to perform update and insert operations. This means that in the system, once a student or an instructor inserts a new assessment in the remote database residing on their device, the new entry is replicated to all other databases in the system and becomes accessible by all other users upon syncing.

The backbone messaging communication between Apps and Core is implemented using the MQTT framework [7]. MQTT is a well-recognized publish-subscribe framework of IoT that uses a highly scalable message broker to allow publishers to publish topics and for subscribers to subscribe to patterns of topics. A topic is a simple hierarchy that is typically domain-specific.

A. Instructor App

Once the instructor logs into the Instructor App, they can proceed to create a new assessment like a quiz for a particular class and add test items to the quiz from a precompiled list of

questions stored in the NSQL database. When the instructor is done, as workflow (1) in Fig. 1 shows, the quiz is automatically published to students enrolled in the class via the MQTT broker.

B. Student App

When a student logs into the Student App, they automatically subscribe to the MQTT topics marked for their class and receive all the quizzes published by their instructor. One advantage of using MQTT is that rather than waiting for the student to complete a quiz, the result of each question is published as soon as the student submits an answer, optionally along with context information collected from their mobile device. The context information can include readings of the in-built sensors (e.g., location, temperature etc.) on the mobile device which may be used later for analytics. The JSON file carrying the student answer and the context information is sent to the core via the MQTT broker as shown in workflow (2) in Fig. 1.

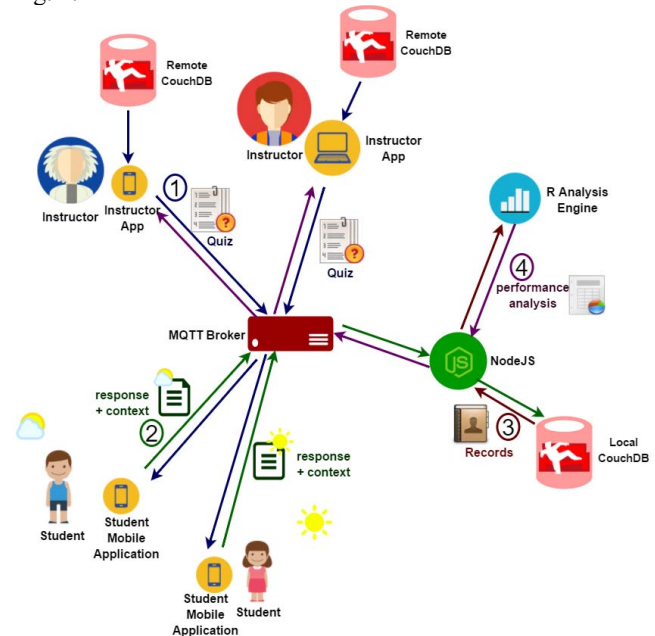


Fig. 1. System Architecture

C. The Core

The sole subscriber to all the student attempts is the Core module, which consists of a NodeJS (<https://nodejs.org/en/>) server that communicates with CouchDB and an analytic engine like R (<https://www.r-project.org/>). NodeJS is an event-driven server-side application runtime environment. While this platform does not natively support multi-threading, the server uses a thread pool where blocking processes can run and notify the main thread once they are done, via callbacks. This enables the NodeJS server to perform asynchronous I/O, and provides high throughput and scalability in web and real-time applications. Another advantage of NodeJS is that it enables the use of a single programming language for all aspects of the Web-based distributed application; Cordova

uses Html5 and JavaScript, and CouchDB uses JavaScript for defining data views and filter criteria.

Once the Core receives a results data from a student, the report is appended to the student's record in CouchDB, and the analysis engine is notified of a new entry. Core uses a Restful architecture to communicate with the analytic engine. When the analytic engine is ready to execute, it sends an HTTP GET request to the NodeJS server, upon which the latter retrieves the appropriate student records from CouchDB and converts them to a CSV file that is sent back to the analysis engine as a GET response as shown in workflow (3) in Fig 1. In workflow (4), the analytic engine (e.g., R) sends back the results of the analysis to NodeJS in the form of a PDF file, which NodeJS then publishes to instructors via MQTT. The Instructor App allows instructors to view this PDF file as soon as it is received.

As Fig. 2 shows, the Core node consists of three primary elements: NodeJS application, a Local CouchDB, and the R Analysis engine. As Fig. 2 shows, the NodeJS application is responsible for various tasks including receiving reports of all students in the system via MQTT, communicating with the database, functioning as an HTTP client and server to communicate with the analysis engine, and publishing the analysis files to their perspective topics via MQTT in real time.

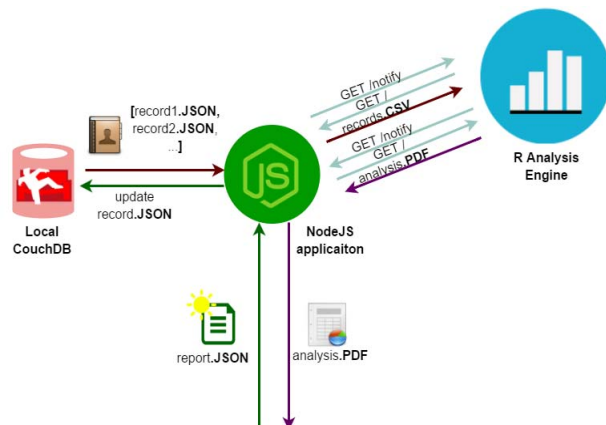


Fig. 2. Core node components

The Core server is also responsible for formatting and modifying information as it flows from one point to another. For example, once assessment results have been received as a JSON file, NodeJS application updates the record in the database, sends a GET request to notify the R analysis engine, and becomes ready to retrieve the records as an array of JSON files from the database and convert them to a single CSV file if the analytic engine requests it, and finally publish the PDF once the analytic engine produces it. The Local CouchDB stores a JSON representation of the curriculum, the questions in QTI format, as well as the records of all students registered in the system. It should be mentioned that different analytic engines can be easily incorporated into this architecture depending on requirements of the implementation environment. Since communication between the server and the analytic engine takes place over HTTP using a Restful architecture, incorporation of various analytic engines is

convenient and straight forward. In addition, for a particular implementation, the system can integrate various analytic engines that produce different results. The server can then publish these varied analysis files to different topics that teachers and administrators subscribe to depending on their interest.

Different types of reports can be generated based on the CSV file received by the analytical engine. These reports are distributed to teachers, principals, and administrators who subscribe to different topics depending on their interest. The sole publisher of reports based on these topics is the Core, which publishes reports as they are received from the R engine. For example, say a student number 14000009 answers a specific question from the learning outcome “Reading Comprehension,” the analytic engine generates new reports that show the student’s overall performance in every Learning Outcome (LO) so far, the student’s performance in that LO compared to their classmates, and their performance compared to all other students who have answered question related to their location and context. Each report is published by the Core server to a different MQTT topic, and all teachers and principals have the option to subscribe to one or more of these topics.

Finally, in order to ensure that the system can easily be integrated into existing learning systems, the questions used in the systems follow a JSON-ized form of the Question and Test Interoperability (QTI) format (<http://www.imsglobal.org/question>) which is one of the most widely-used standards for representing online assessments. The XML-based format was developed by the IMS Global Learning Consortium to facilitate the exchange of assessment items between different learning tools. The format supports a variety of question types including multiple-choice questions, essay questions, and interactive questions where student can answer by moving graphic objects around the page.

III. EVALUATION

Two sets of experiments were used to evaluate the system. In the first experiment the Student and Instructor mobile Apps were installed on two Samsung smart phones running Android. All the backend and middleware software including the NodeJS server, R engine, CouchDB and the MQTT Broker (HiveMQ) ran on a conventional laptop running Windows 8.1. Typical scenarios where an instructor created and posted questions, and a student retrieved and answered the question were simulated. This experiment was primarily used to monitor the performance of the mobile Apps in terms of power consumption under typical circumstances. The results are shown in Fig. 3. As Fig. 3 shows, continuously utilizing the Apps for 5 minutes leads to a slight increase in power consumption. The increase in power consumption can primarily be attributed to continuously logging in, which requires making HTTP requests to the NodeJS server. Furthermore, when it comes to creating a quiz, the instructor is required to choose a learning outcome, based on which questions under that outcome are retrieved from CouchDB through NodeJS via HTTP requests as well. Other minor

contributors are the publication of quizzes, subscription to those quizzes and downloading of analysis reports. However, the 1.5% reduction in battery level when using the App is within acceptable norms.

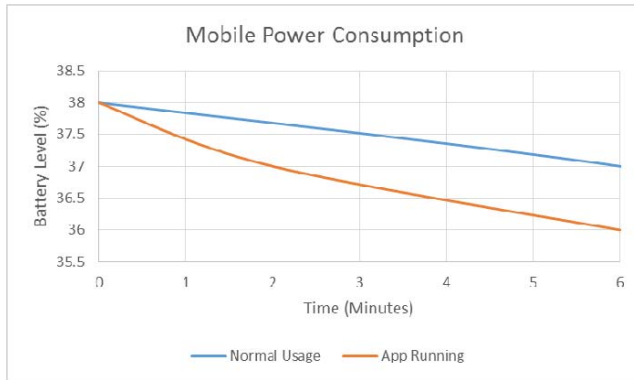


Fig. 3. Power consumption of Instructor and Student Apps

For the second experiment, four personal computers were used. The backend software including NodeJS server, R engine, and CouchDB ran on one typical Windows 8.1 personal computer. The MQTT broker HiveMQ ran on a separate machine. The students were emulated by NodeJS clients running on two additional machines.

Each experimental run consisted of one teacher, and a number of students attempting to attempt an assessment simultaneously. The number of students simultaneously attempting an assessment was varied from 10 students to 100 students. The Wireshark (<https://www.wireshark.org/>) software was used to acquire the average throughput and packet loss values for each scenario. The results from the average throughput in bits/seconds are shown in Fig. 4.

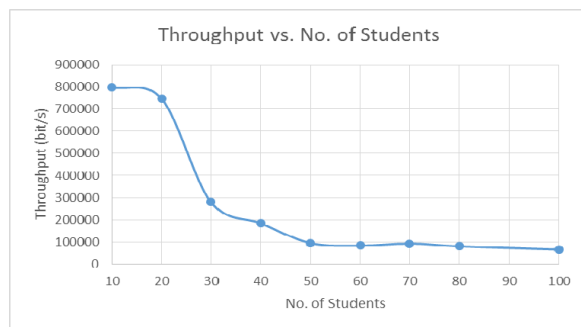


Fig. 4. Average throughput vs. number of students attempting assessments at the same time

As expected, the throughput decreased with the increase in number of students. The results show that a conventional laptop can easily handle up to 20 concurrent students without any decrease in throughput. However, while there is a big drop between the case of 10 students and 40 students, the system seems to reach a steady state afterwards. Additional

simulations are required in the future to confirm this pattern. In terms of packet loss, on the other hand, the system experienced none, even for 100 students. Since MQTT provides retransmission and loss mitigation mechanisms, one can safely assume the system would scale for a larger number of students.

IV. CONCLUSION

The use of mobile devices as a part of holistic educational interventions that involve teacher training and mentoring, and use of summative and formative assessments to improve governance and classroom teaching practice have resulted in small to large statistical effects. It is expected that the proposed system, due to higher transparency and real-time visibility, will result in higher learning effect sizes in future educational interventions that require mobile-based formative assessment as a key component for their theory of change.

The mobile-based system suggested in this paper allows teachers to collect assessments incrementally by using a single tablet or mobile phone. Both real-time response, and context awareness allows educators to have a better understanding of the knowledge gaps students may suffer from and accordingly modify their teaching methodologies and educational policies. Thus, the students' learning experience is greatly enhanced. The results of such assessments being conducted across a school system or a region are dynamically integrated and, as a result, on-demand statistical and analytic reports can be made available to various stakeholder in the educational system. This is achieved through the usage of a NodeJS server that communicated with CouchDB, an MQTT broker, and an R analytic engine. The utilization of MQTT messaging protocol was the key element that allowed for real-time response to take place.

REFERENCES

- [1] Tangerine. [Online]. Available: <http://www.tangerinecentral.org/>. [Accessed: 11-Jun-2016].
- [2] I. A. Zuolkernan, S. Lutfeali, and A. Karim, "Using tablets and satellite-based internet to deliver numeracy education to marginalized children in a developing country," in Global Humanitarian Technology Conference (GHTC), 2014 IEEE, 2014, pp. 294–301.
- [3] I. A. Zuolkernan and A. Karim, "Using Learning Technologies to Enhance Numeracy Competence in Rural Public Schools," in ICT in Education in Global Context, Springer, 2016, pp. 253–276.
- [4] S. B. Shum, S. Knight, and K. Littleton, "Learning analytics," in UNESCO Institute for Information Technologies in Education. Policy Brief, 2012.
- [5] R. Zakaria and I. A. Zuolkernan, "Using Knowledge Space theory to Personalize Teaching for Groups of Students," in Advanced Learning Technologies (ICALT), 2015 IEEE 15th International Conference on, 2015, pp. 58–60.
- [6] Apache CouchDB. [Online]. Available: <http://couchdb.apache.org/>. [Accessed: 10-Jun-2016].
- [7] MQTT Version 3.1.1, 2014. [Online]. Available: <http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>. [Accessed: 01-Jun-2016]